Итоговый проект
Трек А. Реляционная база данных

Проектирование учебной базы данных интернет-магазина электроники

Название проекта	Интернет-магазин электроники: реляционная база данных для анализа продаж и покупательской активности
Автор	Грошников Евгений Сергеевич
Тип проекта	Трек А. Реляционная база данных
Источник данных	Синтетический учебный набор данных
Объём данных	50 покупателей, 8 категорий, 40 товаров, 150 заказов, 300 позиций заказов
Инструменты	SQLite, SQL, DBeaver, Python, Matplotlib

Санкт-Петербург, 2026

1. Предметная область и цель проекта

Проект посвящён учёту и анализу продаж интернет-магазина электроники. Модель хранит сведения о покупателях, категориях и товарах, заказах и их составе. Она позволяет оценить выручку, популярность товаров и повторные покупки без дублирования справочных данных в каждом заказе.

Цель работы — спроектировать реляционную базу SQLite, заполнить её согласованным учебным набором, подготовить более пяти SQL-запросов и визуализировать выручку по категориям и месяцам. В завершённых заказах за исследуемый период выручка составляет 5 363 486,00 руб. Наибольший вклад в неё вносят ноутбуки и смартфоны.

2. Источники данных и предварительная подготовка

Данные синтетические: имена покупателей, названия товаров и продажи не описывают деятельность реальной компании.

Набор представлен в трёх формах: CSV-файлы в data/csv/, SQL-вставки в sql/insert_data.sql и готовая база data/online_store.sqlite. Для восстановления базы используются create_tables.sql и insert_data.sql. CSV служат переносимым представлением тех же таблиц.

Таблица	Содержание	Строк
customers	Покупатели и даты регистрации	50
categories	Категории товаров	8
products	Каталог и текущие цены	40
orders	Заказы, даты и статусы	150
order_items	Товары и цены в заказах	300

Цены хранятся целыми числами в копейках. Текущая цена товара находится в products.price_kopecks, а цена конкретной продажи — в order_items.unit_price_kopecks. Анализ выручки использует цену из позиции заказа, поэтому изменение каталога не меняет историю продаж.

Статус заказа	Значение	Заказов
completed	Завершён	126
pending	Ожидает завершения	4

Статус заказа	Значение	Заказов
cancelled	Отменён	20

В расчёты выручки, проданных единиц и среднего чека включаются только completed. Отменённые и ожидающие заказы в эти показатели не входят.

3. ER-диаграмма и связи

Схема включает пять таблиц. Покупатель может иметь несколько заказов, категория иметь несколько товаров, а заказ — несколько позиций. Таблица order_items связывает заказы и товары, реализуя связь «многие ко многим».

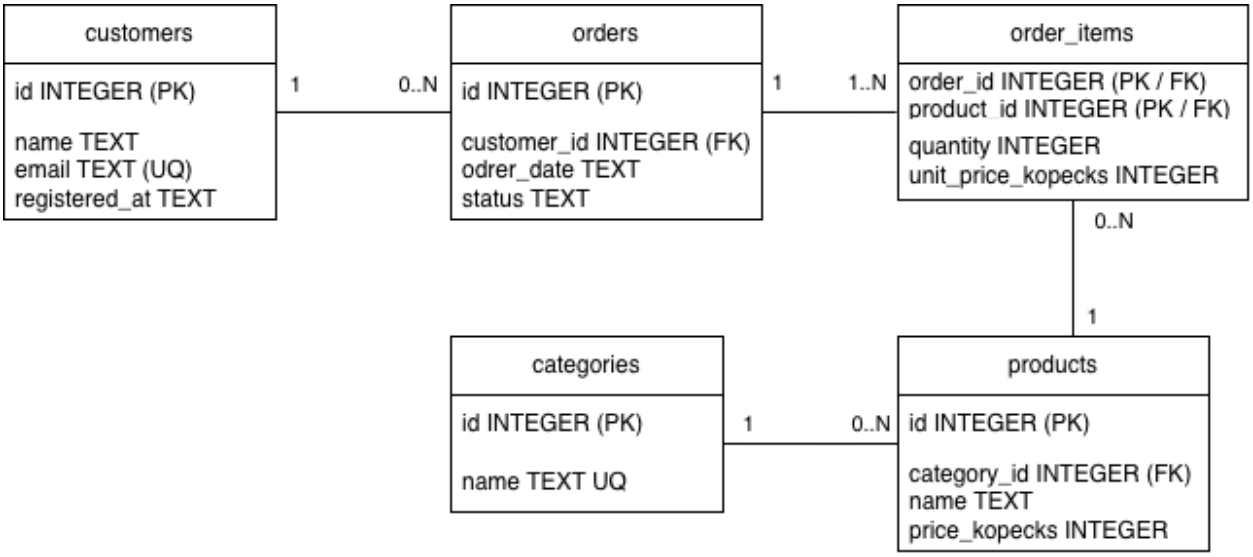


Рисунок 1. ER-диаграмма базы интернет-магазина

4. Описание таблиц и ограничений

Таблица	Ключи и основные атрибуты
customers	id (первичный ключ); name, email, registered_at; email уникален.
categories	id (первичный ключ); name (уникальное название категории).
products	id (первичный ключ); category_id (внешний ключ); name,
orders	id (первичный ключ); customer_id (внешний ключ); order_date, status.

Таблица	Ключи и основные атрибуты
order_items	Составной первичный ключ (order_id, product_id); оба поля — внешние ключи; quantity, unit_price_kopecks.

Таблицы созданы в режиме STRICT. Ограничения NOT NULL, UNIQUE и CHECK проверяют обязательные значения, уникальность, положительные цены и количество, допустимые статусы и формат дат. Внешние ключи включаются через PRAGMA foreign_keys = ON для каждого соединения, изменяющего данные.

Справочники вынесены отдельно: название категории не повторяется у каждого товара, а данные покупателя — у каждого заказа. Составной ключ order_items не допускает повторения одного товара в одном заказе; количество хранится в quantity.

Удаление заказа каскадно удаляет его позиции. Удаление связанных покупателей, категорий и товаров ограничено RESTRICT. Для соединений созданы индексы по products.category_id, orders.customer_id и order_items.product_id.

На диаграмме заказ содержит одну или несколько позиций. Это правило соблюдается в текущем наборе, но отдельного SQL-ограничения, запрещающего заказ без позиций, в схеме нет. Проверка набора не обнаружила пустых заказов.

5. SQL-запросы и примеры результатов

Файл sql/queries.sql содержит восемь именованных запросов. В них выполнена фильтрация, сортировка, соединения таблиц, агрегирование, HAVING, подзапросы и обобщённые табличные выражения.

Запрос	Аналитическая задача
Q1	Найти товары с текущей ценой выше 50 000 руб.
Q2	Посчитать выручку по всем категориям, включая категории без продаж.
Q3	Вывести топ-10 товаров по количеству проданных единиц.
Q4	Найти покупателей с более чем тремя завершёнными заказами.
Q5	Посчитать выручку по месяцам.
Q6	Посчитать средний чек завершённого заказа по месяцам.

Запрос	Аналитическая задача
Q7	Найти покупателей без заказов любого статуса.
Q8	Найти категории с выручкой выше среднего по всем категориям.

5.1 Выручка по категориям

Категория	Выручка, руб.	Доля, %
Ноутбуки	2 292 676,50	42,7
Смартфоны	1 063 495,00	19,8
Планшеты	664 032,50	12,4
Мониторы	608 752,00	11,3
Наушники	385 586,50	7,2
Клавиатуры	158 449,00	3,0
Мыши	103 849,00	1,9
Аксессуары	86 645,50	1,6

Итого: 5 363 486,00 руб. Ноутбуки и смартфоны вместе дают 62,6% выручки. В Q2 фильтр completed расположен в условии соединения, чтобы категории без завершённых продаж сохранялись в результате с нулевой суммой.

5.2 Товары дороже 50 000 рублей

Q1 возвращает восемь товаров. Самая высокая текущая цена у Lumen Studio 16: 119 990,00 руб., далее идут Lumen Ultra 512 ГБ: 99 990,00 руб. и Vektor Game 15: 94 990,00 руб.

5.3 Популярные товары и покупатели

Q3 ранжирует товары по сумме quantity в завершённых заказах. Ниже показаны первые пять строк из десяти результатов.

Товар	Продано, шт.
Кабель USB-C 1 м	25
Nord Mouse Wireless	23

Товар	Продано, шт.
Nord Buds Lite	20
Vektor Book 14	18
Nord S10 128 ГБ	15

По Q4 двенадцать покупателей имеют более трёх завершённых заказов. Лидер — Михаил Соколов с 11 заказами; у Марии Соколовой и Ивана Соколова по 8. Заказы считаются без соединения с позициями, поэтому один заказ не учитывается несколько раз.

Q7 находит десять покупателей без заказов любого статуса: идентификаторы 41–50. Это 20% всех регистраций. Результат показывает отсутствие заказов, не объясняет причины неактивности.

5.4 Помесячная выручка и средний чек

Месяц 2026 года	Заказов	Выручка, руб.	Средний чек, руб.
Май	17	1 304 145,50	76 714,44
Июнь	19	579 379,50	30 493,66
Июль	21	744 179,50	35 437,12
Август	23	1 002 428,00	43 583,83
Сентябрь	24	941 387,00	39 224,46
Октябрь	22	791 966,50	35 998,48

Максимальная выручка приходится на май — 1 304 145,50 руб., минимальная на июнь — 579 379,50 руб. В сентябре завершённых заказов больше всего: 24, однако выручка ниже майской. В мае средний чек составляет 76 714,44 руб., в сентябре 39 224,46 руб.

В Q6 CTE сначала вычисляет сумму каждой покупки, затем AVG усредняет суммы заказов. Такой порядок исключает ошибку, при которой средний чек подменяется средней стоимостью товарной позиции.

5.5 Категории выше среднего

Средняя выручка по восьми категориям составляет 670 435,75 руб. Q8 возвращает ноутбуки и смартфоны. Планшеты с выручкой 664 032,50 руб.

находятся чуть ниже этого порога. Среднее включает все категории, в том числе категории с нулевой выручкой, если они появятся в базе.

6. Визуализация результатов

Скрипт visualize_revenue.py читает результаты Q2 и Q5 непосредственно из SQLite и сохраняет графики в PNG и SVG. Перед построением он проверяет, что сумма выручки по категориям совпадает с суммой по месяцам.



Рисунок 2. Выручка по категориям за май–октябрь 2026 года

Столбцы упорядочены по убыванию выручки. Две ведущие категории выделены синим. Шкала начинается с нуля, суммы представлены в миллионах рублей.

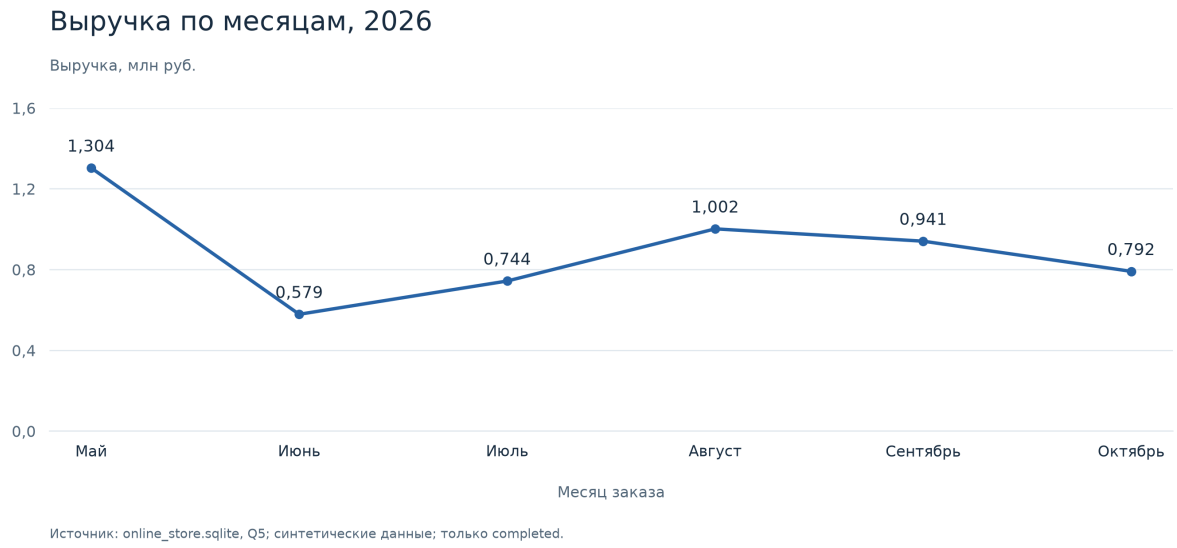


Рисунок 3. Выручка по месяцам за 2026 год

Линия показывает снижение после мая, восстановление к августу и последующее уменьшение. Подписи на графиках округлены; точные значения приведены в таблицах отчёта.

7. Выводы

Спроектирована база из пяти связанных таблиц. Разделение справочников и операций сокращает повторение данных, а цена в позиции заказа сохраняет стоимость конкретной продажи. Восемь SQL-запросов охватывают анализ каталога, продаж и покупательской активности.

В 126 завершённых заказах продано 288 единиц товаров на 5 363 486,00 руб. Ноутбуки приносят 2 292 676,50 руб., смартфоны — 1 063 495,00 руб. Их общая доля составляет 62,6%. При этом лидер по количеству проданных единиц — кабель USB-C 1 м: 25 штук. Лидеры по объёму продаж и по выручке различаются.

Май лидирует по выручке и среднему чеку, хотя максимум числа завершённых заказов приходится на сентябрь. Для интерпретации динамики полезно рассматривать выручку, число заказов и средний чек вместе.

8. Проверки и воспроизводимость

Проверки PRAGMA integrity_check и PRAGMA foreign_key_check подтверждают целостность текущей базы и отсутствие нарушений внешних ключей. Данные всех пяти таблиц совпадают с CSV и базой, восстановленной

из SQL. Дополнительно не найдены заказы без позиций и заказы, оформленные до регистрации покупателя.

Для повторного анализа достаточно открыть data/online_store.sqlite в DBeaver и выполнить sql/queries.sql. При восстановлении сначала выполняется sql/create_tables.sql, затем sql/insert_data.sql. Скрипт создания удаляет существующие таблицы, поэтому изменения в базе перед восстановлением следует сохранить отдельно.

Для графиков нужны Python 3.11+ и зависимости из requirements.txt. После установки зависимостей команда python visualize_revenue.py создаёт четыре файла в output/. Подробные команды приведены в README.md. Проверенный запуск обработал 8 категорий и 6 месяцев без ошибок.

9. Решения и ограничения

1. Отдельные сущности и позиции заказа. Разделены покупатели, категории, товары, заказы и позиции заказа, потому что у них разное назначение. Покупатель может оформить несколько заказов, а товар встретиться в разных заказах. Таблица order_items связывает товары и заказы, не заставляя повторять данные покупателя и дату покупки для каждой позиции. Пара (order_id, product_id) выбрана ключом, чтобы один товар занимал одну строку заказа, а число единиц задавалось quantity. Это упрощает модель, но не позволяет записать один товар в одном заказе двумя строками с разными ценами.

2. Цена продажи отдельно от цены каталога. unit_price_kopecks хранится отдельно в позиции заказа, поскольку текущая цена товара может измениться. Если пересчитывать старые продажи по products.price_kopecks, прошлые суммы будут зависеть от сегодняшнего каталога. Копия цены здесь сохраняет факт сделки. Целые копейки позволяют хранить суммы без дробной части и точно считать произведения и суммы в пределах диапазона INTEGER. Перевод в рубли нужен уже для представления результатов.

3. Ограничения на уровне базы. Используются внешние ключи, чтобы позиция не ссылалась на несуществующий заказ или товар. CHECK не допускает отрицательных цен и количества, UNIQUE защищает от повторения email, а STRICT и NOT NULL проверяют типы и обязательные поля. Такие правила действуют и при ручном импорте через DBeaver, поэтому проверок одного Python-скрипта недостаточно. Для внешних ключей важно включать PRAGMA foreign_keys = ON в каждом соединении. При

удалении заказа его позиции удаляются каскадно, а удаление связанных справочных записей запрещено.

4. SQLite для учебного проекта. Выбран SQLite, потому что базу можно передать одним файлом и открыть без отдельного сервера. Для пяти таблиц и демонстрации SQL этого достаточно. Скрипты создания и заполнения позволяют восстановить набор и проверить результат на другом компьютере. Это решение рассчитано на локальный анализ. Для магазина с большим числом одновременных записей, отдельными правами пользователей и серверным обслуживанием стоило бы рассмотреть другую СУБД.

5. Единое определение выручки. В выручку выключены только completed заказы, чтобы ожидающие и отменённые заказы не увеличивали сумму состоявшихся продаж. Во всех связанных запросах применяется одно правило: количество умножается на цену из позиции заказа. Так результаты по категориям и месяцам можно сопоставлять. При этом completed это принятое в проекте условие учёта продажи, а не подтверждение движения денег: даты оплаты и поступления средств в модели нет.

Что делают ключевые запросы и код

Q2, вклад категорий в выручку. Запрос соединяет категории с товарами, позициями и заказами, затем суммирует стоимость завершённых продаж по каждой категории. LEFT JOIN и фильтр статуса в ON сохраняют категории без таких продаж; CASE и COALESCE дают им нулевую сумму. Фильтр в WHERE исключил бы эти категории. Результат показывает, что ноутбуки принесли 2 292 676,50 руб., а вместе со смартфонами дали 62,6% выручки. Это вклад в денежную сумму, а не показатель прибыльности.

Q6, средняя сумма заказа. CTE order_totals сначала объединяет позиции в сумму одного завершённого заказа, затем AVG усредняет эти суммы по месяцам. Без первого уровня расчёт характеризовал бы стоимость позиции, а не покупки целиком. В мае средний чек равен 76 714,44 руб. при 17 заказах, в сентябре 39 224,46 руб. при 24 заказах. Поэтому большее число заказов само по себе не означает большую выручку.

main() — проверка согласованности перед визуализацией. Скрипт получает Q2 и Q5 из одного SQL-файла и сравнивает суммы по категориям и месяцам до сохранения графиков. Обе суммы равны 5 363 486,00 руб. Если они разойдутся, построение остановится с ошибкой. Эта проверка помогает обнаружить различия в фильтрах или соединениях, но совпадение итогов не

доказывает достоверность данных: одна и та же ошибка может присутствовать в обоих расчётах.

Границы выводов и возможные улучшения

Данные и смысл показателей. Набор данных охватывает только шесть месяцев. По нему нельзя оценивать реальный рынок электроники, доказывать сезонность или связывать падение выручки с конкретной причиной. Нет данных о рекламе, остатках, себестоимости, возвратах и расходах, поэтому нельзя рассчитать прибыль и объяснить изменения спроса. Месяц определяется датой заказа, а не оплаты. Для таких задач нужны реальные наблюдения за более длительный период и дополнительные поля.

Полнота модели и проверок. В текущем наборе каждый заказ имеет позиции и не предшествует регистрации покупателя, но схема сама эти правила не гарантирует. Можно было бы добавить проверки при загрузке или триггеры, чтобы новые данные не нарушали их. Для разных цен одного товара внутри заказа понадобился бы отдельный идентификатор позиции. Автоматическая сверка SQL, CSV и SQLite помогла бы предотвратить расхождение трёх представлений после обновления.

Точность расчётов и пропущенные месяцы. Деление на 100.0 в SQL переводит результат в REAL; последующий Decimal в Python не восстанавливает уже потерянную точность. Для больших сумм нужно сохранять целые копейки до вывода. Функция complete_months заполняет пропуски внутри периода нулями. Это допустимо при полной выгрузке, когда отсутствие строк означает отсутствие продаж; при неполной загрузке ноль нужно отличать от неизвестного значения.

10. Материалы

Репозиторий со всеми файлами по проекту, включая декларацию использования ИИ, размещен по адресу: https://github.com/ZhenyaGro/ITMO-AI-Data-Analysis-proj